

29 de Mayo del 2026

UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

Comparación de Modelos de Redes Neuronales LSTM y Arboles de Decisión para la Identificación y Clasificación de Secuencias de Movimiento de Baile (Dance Labo)

Materia: Computo Bioinspirado

Docente: Leticia Cervantes Huerta

Alumnos: Ainy Contreras Mendoza

Daniel Tornero Solano

Juan Carlos Cruz Hernandez

Jorge Alberto Ruiz Rodriguez

Josue Fernando Robledo Zuñiga

Grupo: 564

2. Índice

3. Resumen

4. Introducción

4.1 Contexto del Proyecto

4.2 Planteamiento del Problema

4.3 Alcance del Proyecto

4.4 Estructura del Documento

5. Objetivos

5.1 Objetivo General

5.2 Objetivos Específicos

6. Justificación

7. Marco Teórico

7.1 Análisis y Reconocimiento de Movimiento Humano

7.2 Visión Artificial: Detección YOLOv8 y Estimación MediaPipe Pose

7.3 Redes Neuronales Recurrentes (RNN) y Long Short-Term Memory (LSTM)

7.4 Árboles de Decisión (Decision Trees)

7.5 Criterios y Métricas de Comparación de Modelos de Inteligencia Artificial

8. Metodología

8.1 Descripción y Estructura del Conjunto de Datos

8.2 Preprocesamiento y Normalización del Movimiento

8.3 Diseño e Implementación del Modelo LSTM

8.4 Diseño e Implementación del Modelo de Árbol de Decisión

8.5 Protocolo Experimental de 20 Corridas Comparativas

8.6 Entorno de Ejecución y Herramientas Tecnológicas

8.7 Métricas de Evaluación Cuantitativa

9. Resultados

9.1 Tabla Consolidada de 20 Corridas Comparativas

9.2 Análisis y Comparación Cuantitativa

9.3 Análisis Cualitativo Detallado

9.4 Interpretación del Rendimiento de los Modelos y Estabilidad

10. Discusión y Conclusiones

11. Referencias

12. Anexos

12.1 Anexo A: Código Fuente de Modelos y Comparación

12.2 Anexo B: Capturas de Pantalla e Interfaces de la Aplicación

12.3 Anexo C: Documentación Técnica de Integración y Red (UDP/OSC)

3. Resumen

Este proyecto presenta el desarrollo, validación e integración de **Dance Labo**, un sistema interactivo e inteligente enfocado en la captura, recreación tridimensional y evaluación de secuencias de baile en tiempo real. El núcleo del software combina algoritmos de visión artificial de última generación (detección de personas con YOLOv8 y estimación de pose 3D con MediaPipe Pose) con modelos de aprendizaje supervisado y una interfaz interactiva de renderizado en el motor gráfico Unity. El objetivo principal de esta investigación consiste en evaluar, justificar y comparar de manera cuantitativa y cualitativa dos enfoques clasificatorios para series de datos temporales esqueléticos: un clasificador basado en **Redes Neuronales Recurrentes del tipo Long Short-Term Memory (LSTM)** y un clasificador de aprendizaje automático clásico basado en **Árboles de Decisión (Decision Trees)**.

La metodología experimental comprende la adquisición de un conjunto de datos real compuesto por 26 secuencias de movimiento enfocadas en un movimiento coreográfico unificado de referencia (Secuencia1). Los datos espaciales crudos son mapeados a 9 articulaciones del estándar humanoid (Mixamo) y sometidos a un preprocesamiento de interpolación lineal y *padding* uniforme a 150 frames. Los modelos se entrenaron bajo un protocolo de 20 variaciones aleatorizadas de divisiones de datos (*splits*) e hiperparámetros del modelo.

Los resultados cuantitativos demuestran que la red neuronal **LSTM superó de manera consistente en exactitud (Accuracy promedio: 75.14%)** al clasificador de **Árbol de Decisión (Accuracy promedio: 65.43%)** en datos no vistos de validación. Sin embargo, el Árbol de Decisión exhibió un tiempo de entrenamiento extremadamente bajo (0.055 s promedio) en comparación con el modelo LSTM (17.95 s promedio). La discusión técnica concluye que el modelo LSTM es significativamente superior para modelar dependencias secuenciales complejas y variaciones temporales de velocidad en el movimiento de baile, logrando integrarse con éxito en el pipeline de evaluación interactivo de Unity a frecuencias mayores a 30 FPS en CPU.

Palabras clave: Aprendizaje Máximo, LSTM, Árboles de Decisión, MediaPipe, Captura de Movimiento, Baile, Visión Artificial, Unity.

4. Introducción

4.1 Contexto del Proyecto

El análisis automatizado del movimiento humano a través de tecnologías de visión computacional y aprendizaje profundo (Deep Learning) ha experimentado una revolución exponencial en la última década. La capacidad de discernir, seguir y calificar la corporalidad humana sin la necesidad de trajes especializados o marcadores ópticos (captura markerless) ha democratizado campos como la telerehabilitación, el entrenamiento atlético, los videojuegos interactivos y la expresión artística de la danza.

El baile, como disciplina física altamente estructurada, expresiva y rítmica, representa un desafío de gran complejidad para la inteligencia artificial. A diferencia de las posturas estáticas, el baile se compone de dinámicas espaciotemporales intrincadas, variaciones de velocidad rítmica, y transiciones fluidas de múltiples segmentos del cuerpo. El desarrollo de sistemas capaces de asistir a estudiantes proporcionando retroalimentación correctiva inmediata en sus hogares u aulas requiere soluciones robustas que unan la computación en tiempo real y la predicción secuencial.

4.2 Planteamiento del Problema

Para implementar un motor de evaluación en tiempo real de baile, es imperativo contar con un modelo de clasificación capaz de identificar con precisión qué movimiento de referencia está intentando realizar el usuario. En el diseño de este sistema, surge un dilema arquitectónico clásico en la Inteligencia Artificial Aplicada:

¿Es preferible emplear una arquitectura secuencial profunda o un modelo clásico de aprendizaje supervisado?

Las **Redes Neuronales Recurrentes (RNN)**, en particular las arquitecturas **LSTM (Long Short-Term Memory)**, están diseñadas intrínsecamente para procesar secuencias temporales, manteniendo memoria de los frames anteriores para clasificar el movimiento en su conjunto. Por otro lado, los **Árboles de Decisión (Decision Trees)** representan un enfoque tradicional, altamente interpretable y computacionalmente eficiente, pero que carece de una noción intrínseca de

secuencia temporal a menos que los datos sean aplanados (*flattened*), perdiendo la continuidad temporal directa del movimiento.

Comparar cuantitativamente la exactitud, el error de generalización, la velocidad de entrenamiento y la viabilidad de integración entre estos dos paradigmas (LSTM y Árboles de Decisión) en un conjunto de datos real de baile es fundamental para justificar la selección del modelo de inferencia en el entorno de producción interactivo de Dance Labo.

4.3 Alcance del Proyecto

El alcance de esta investigación comprende:

1. El diseño del pipeline de visión por computadora: detección y recorte de la persona con YOLOv8, y estimación de 33 landmarks 3D con MediaPipe Pose.
2. El mapeo y normalización tridimensional de landmarks a 9 rotaciones del esqueleto virtual del robot (LABO new rig) en Unity.
3. La implementación, optimización y entrenamiento del clasificador generativo LSTM de 3 capas y del Árbol de Decisión.
4. La ejecución de un protocolo formal de 20 corridas experimentales comparativas aleatorizadas para contrastar ambos modelos en términos de tasa de error, exactitud y tiempo de procesamiento.
5. La integración de los resultados de inferencia en tiempo real por medio de una arquitectura de puente de red UDP entre Python (PyQt6) y Unity (C#).

4.4 Estructura del Documento

El reporte se organiza de la siguiente manera: la Sección 5 detalla los objetivos perseguidos; la Sección 6 provee la justificación tecnológica y social del proyecto; la Sección 7 desarrolla las bases conceptuales del marco teórico; la Sección 8 describe la metodología experimental empleada; la Sección 9 expone la tabla de las 20 corridas y el análisis cuantitativo y cualitativo; la Sección 10 presenta la discusión crítica y conclusiones del proyecto; la Sección 11 provee el catálogo bibliográfico formateado en APA y la Sección 12 anexa el código fuente implementado y las evidencias de la interfaz.

5. Objetivos

5.1 Objetivo General

Comparar el rendimiento cuantitativo y cualitativo de un clasificador basado en Redes Neuronales Recurrentes LSTM frente a un clasificador basado en Árboles de Decisión para la identificación y clasificación de secuencias de movimiento en el sistema interactivo de baile Dance Labo.

5.2 Objetivos Específicos

1. Configurar y validar un entorno de ejecución robusto en Python que integre la captura markerless mediante YOLOv8 y MediaPipe.
2. Desarrollar un sistema de mapeo de huesos (MixamoMapper) que normalice las coordenadas espaciales en vectores tridimensionales independientes de la fisiología del usuario.
3. **Diseñar e implementar el modelo de Red Neuronal LSTM** en PyTorch adaptado al flujo de series temporales de 27 dimensiones (9 articulaciones × xyz).
4. **Implementar el modelo de Árbol de Decisión** utilizando Scikit-Learn alimentándolo con los vectores temporales aplanados.
5. Diseñar un script formal de pruebas con **20 corridas experimentales** variando aleatoriamente las divisiones del conjunto de datos y los hiperparámetros para evaluar de forma cuantitativa la exactitud y tiempo de entrenamiento de ambos esquemas.
6. Analizar de manera cualitativa la interpretabilidad y viabilidad de inferencia a 30 FPS de ambos clasificadores para integrarlos al pipeline de feedback de Unity.

6. Justificación

El desarrollo de software educativo apoyado por inteligencia artificial tiene un impacto multifacético:

Impacto Educativo e Inclusivo: En la enseñanza de la danza o terapias físicas rehabilitadoras, el acceso a entrenadores especializados es costoso y geográficamente limitado. Un sistema de captura sin marcadores basado en

cámaras convencionales permite a los alumnos practicar desde sus hogares con guías virtuales detallados. Esto democratiza la educación artística y fomenta hábitos saludables de actividad física de manera autónoma.

Impacto Tecnológico: El proyecto explora la sinergia de herramientas modernas como la detección de personas (YOLOv8), el seguimiento esquelético (MediaPipe), el modelado secuencial de redes neuronales (PyTorch) y la interactividad 3D de Unity. Estudiar y documentar formalmente la comparación entre modelos profundos y tradicionales aporta valor técnico al diseño de sistemas inteligentes interactivos en tiempo real bajo limitaciones de hardware convencionales.

Aporte Científico: El análisis demuestra cómo estructurar y preprocesar series de tiempo del esqueleto humano de forma que modelos simples y profundos puedan ser contrastados equitativamente. La evaluación exhaustiva de 20 corridas comparativas valida la estabilidad estadística del modelo elegido, evitando la selección arbitraria basada en una única ejecución del clasificador.

7. Marco Teórico

7.1 Análisis y Reconocimiento de Movimiento Humano

El análisis del movimiento humano por computadora ha pasado históricamente de costosos laboratorios de captura ópticos equipados con marcadores retroreflectivos e infrarrojos (como Vicon u OptiTrack) a soluciones basadas en sensores de profundidad de bajo costo (como Microsoft Kinect). Sin embargo, el estado del arte actual se sitúa en la captura monocular libre de marcadores (markerless monocular capture), donde algoritmos entrenados con millones de imágenes son capaces de inferir posiciones tridimensionales de las articulaciones a partir de una señal de video RGB estándar.

La evaluación interactiva del movimiento corporal humano en tiempo real requiere la conjunción coordinada de disciplinas de visión computacional y modelos matemáticos de predicción. La captura y análisis esquelético sin marcadores representa un avance sustancial debido a que prescinde de costosos laboratorios equipados con sensores especializados y trajes reflectivos. La democratización de

estas tecnologías en ámbitos escolares o domésticos descansa sobre la factibilidad de utilizar cámaras monoculares de consumo estándar (webcams) operando de manera integrada con modelos clasificadores ágiles.

7.2 Visión Artificial: Detección YOLOv8 y Estimación MediaPipe Pose

El pipeline de visión de Dance Labo implementa una arquitectura híbrida de dos etapas para optimizar el rendimiento y la precisión de la estimación:

YOLOv8 (You Only Look Once): Es una red neuronal convolucional de una sola etapa entrenada para la detección de objetos en tiempo real. En este sistema, YOLOv8 se encarga de localizar al sujeto principal en el frame de video y devolver una caja delimitadora (*bounding box*). Este paso de pre-localización es crucial, ya que aísla al usuario del fondo y suprime el ruido visual exterior (como muebles, sombras o personas secundarias).

MediaPipe Pose: Una vez recortada la región del sujeto detectada por YOLO, este sub-frame se entrega a MediaPipe Pose. MediaPipe utiliza una arquitectura de red convolucional móvil optimizada para inferir los **33 landmarks corporales** en coordenadas 3D. Al operar sobre la región de interés ya depurada por YOLO, MediaPipe incrementa sustancialmente su estabilidad espacial y su inmunidad a oclusiones parciales.

Flujo del pipeline:

[Frame de Cámara RGB] → [YOLOv8 (Recorte)] → [MediaPipe (33 Landmarks 3D)]

7.3 Redes Neuronales Recurrentes (RNN) y Long Short-Term Memory (LSTM)

Las Redes Neuronales Recurrentes (RNN) tradicionales presentan dificultades inherentes al procesar secuencias de datos a largo plazo debido a los problemas de desvanecimiento y explosión del gradiente durante la retropropagación en el tiempo (Backpropagation Through Time). Para solucionar esto, Hochreiter y Schmidhuber (1997) propusieron la celda LSTM (Long Short-Term Memory).

Una celda LSTM introduce el concepto de **estado de celda (Ct)**, el cual actúa como una cinta transportadora que permite que la información fluya sin alteraciones

severas a través de los pasos temporales. La regulación del flujo de información se realiza mediante tres compuertas (*gates*):

1. **Compuerta de olvido (ft):** Decide qué información descartar del estado de celda anterior basándose en el estado oculto anterior (h_{t-1}) y la entrada actual (x_t).
2. **Compuerta de entrada (it):** Decide qué nuevos datos almacenar en el estado de celda.
3. **Compuerta de salida (ot):** Determina el nuevo estado oculto (h_t) que se enviará al siguiente paso temporal y a las capas superiores.

Las ecuaciones que gobiernan el comportamiento de las celdas LSTM son las siguientes:

$$\begin{aligned}f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\ \tilde{C}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\C_t &= f_t * C_{t-1} + i_t * \tilde{C}_t \\o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\h_t &= o_t * \tanh(C_t)\end{aligned}$$

Esta capacidad para capturar dependencias a largo plazo hace que las LSTMs sean ideales para representar movimientos temporales fluidos y coordinados como las danzas.

7.4 Árboles de Decisión (Decision Trees)

Los Árboles de Decisión son modelos no paramétricos de aprendizaje supervisado utilizados tanto para tareas de clasificación como de regresión. Su construcción se basa en una partición jerárquica del espacio de características del conjunto de datos mediante un enfoque codicioso (greedy) de arriba hacia abajo (top-down induction).

El objetivo es seleccionar en cada nodo interno la característica y el valor de corte que mejor divida a los datos de entrenamiento para maximizar la homogeneidad de los subconjuntos resultantes. Las dos funciones de impureza más comunes para clasificación son:

Impureza de Gini: $Gini(D) = 1 - \sum (p_i^2)$

donde p_i representa la probabilidad de que un elemento pertenezca a la clase i en el conjunto D .

Entropía (Información Mutua): $Entropia(D) = - \sum (p_i \cdot \log_2(p_i))$

Los árboles de decisión ofrecen una **excelente interpretabilidad** al poder representarse visualmente como reglas lógicas llanas. Sin embargo, son propensos al sobreajuste (*overfitting*) si crecen demasiado sin restricciones de poda, y su aplicación en series de tiempo requiere transformaciones previas, puesto que tratan cada dimensión de entrada como variables independientes, ignorando el orden estructural intrínseco.

7.5 Criterios y Métricas de Comparación de Modelos de Inteligencia Artificial

La selección del modelo definitivo para su implantación en un sistema real e interactivo debe basarse en un balance de los siguientes criterios:

Criterio	Red Neuronal LSTM	Árbol de Decisión
Exactitud (Accuracy)	Muy alta en secuencias espaciotemporales continuas.	Media-baja debido a la pérdida de estructura temporal tras aplanamiento.
Tiempo de Entrenamiento	Elevado en CPU (segundos o minutos por la retropropagación en el tiempo).	Instantáneo (milisegundos al basarse en ordenamiento y divisiones).
Interpretabilidad	Caja negra (pesos de tensores de difícil análisis intuitivo).	Muy alta (flujograma de decisiones lógicas visuales).
Eficiencia de Inferencia	Requiere recursos moderados de CPU/GPU (multiplicación secuencial de matrices).	Sumamente veloz (comparaciones lógicas directas de variables).

8. Metodología

8.1 Descripción y Estructura del Conjunto de Datos

Para esta investigación se utilizó el conjunto de datos propietario y depurado recopilado por el equipo de **Dance Labo (LABITO)**. Tras un proceso de optimización del rig de huesos y estandarización para garantizar la coherencia absoluta de las articulaciones entre Python y Unity, se eliminaron las secuencias multi-rig obsoletas que presentaban desajustes e inconsistencias de escala.

El conjunto de datos final de alto rendimiento está compuesto por **26 secuencias de movimiento grabadas** con una frecuencia de muestreo de 30 FPS, enfocadas en un movimiento coreográfico unificado de referencia:

1. Secuencia1 (26 tomas de movimiento secuencial completo)

Cada secuencia tiene una longitud promedio de **150 frames** (~5 segundos de ejecución fluida de baile). Las muestras consisten en archivos de intercambio JSON estructurados bajo el estándar de Mixamo, conteniendo una cabecera con el **movement_name** y una lista de frames numerados. En cada frame, se registran los vectores unitarios de dirección 3D correspondientes a los 12 huesos clave.

JSON de Secuencia (data/sequences/baile.json)

```
"movement_name": "Secuencia1",
"frames": [
  {
    "frame": 0,
    "landmarks": {
      "mixamorig:LeftArm": {
        "x": 0.4945309194401052,
        "y": -0.869160036884879,
        "z": 0.0
      },
      "mixamorig:RightArm": {
        "x": -0.07418944522372485,
        "y": -0.9972441657976225,
        "z": 0.0
      },
      ... (9 huesos adicionales)
```

8.2 Preprocesamiento y Normalización del Movimiento

La entrada del sensor MediaPipe es mapeada por el componente **mixamo_mapper.py** del bridge de Python. Este componente extrae las posiciones relativas tridimensionales del esqueleto y las traduce al sistema de coordenadas de Unity:

```
V_Unity = [ X_MediaPipe, -Y_MediaPipe, -Z_MediaPipe ]
```

Este mapeo a cuaterniones locales y vectores de dirección unitarios aísla las proporciones físicas de la persona (altura, grosor de extremidades), garantizando que el modelo sea entrenado únicamente en base al estilo del movimiento y no a la fisionomía del bailarín.

Puesto que las grabaciones difieren en longitud (desde 200 hasta 600 frames), el preprocesamiento unifica todas las muestras a un tamaño de secuencia uniforme de **T = 150 frames** (~5 segundos a 30 FPS). Las secuencias más cortas son rellenadas con ceros (*zero-padding*) al final, y las secuencias más largas se truncan simétricamente.



8.3 Diseño e Implementación del Modelo LSTM

El clasificador neuronal se codificó en PyTorch. Consta de una capa de entrada secuencial de 150×27 características, conectada a una celda LSTM profunda de dos capas con un tamaño oculto (*hidden size*) de 32 o 64 neuronas según la corrida. La clasificación final se realiza proyectando el estado oculto del último frame temporal (h150) mediante una capa lineal densa hacia la capa de salida que discrimina la correspondencia con la coreografía de referencia (**Secuencia1**).

El entrenamiento empleó el optimizador **Adam** con una tasa de aprendizaje adaptativa gobernada por un planificador *StepLR* (reduciendo a la mitad el valor de LR cada 50 épocas). La función de pérdida utilizada fue la **Entropía Cruzada**

(Cross-Entropy Loss), ideal para la optimización de múltiples categorías exhaustivas.

```
[ Tensor de Entrada (B, 150, 27) ]  
    |  
[ LSTM de 1-2 Capas ] → [ DropOut 20% ]  
    |  
[ Estado Oculto Final h_150 ]  
    |  
[ Capa Densa (FC) ] → [ CrossEntropyLoss ]  
    |  
[ Score por Clase ]
```

8.4 Diseño e Implementación del Modelo de Árbol de Decisión

Dado que el árbol de decisión no procesa datos espaciotemporales de forma nativa, cada muestra de 150×27 valores se aplanó en un vector unidimensional de **4,050 variables independientes** ($150 \times 27 = 4,050$ features). El clasificador se entrenó variando la profundidad máxima (*max_depth*), permitiendo al algoritmo encontrar umbrales de rotación en frames clave para discernir entre los bailes.

Se empleó el paquete **sklearn.tree.DecisionTreeClassifier** variando el parámetro de impureza (**criterion="gini"** y **criterion="entropy"**) y la profundidad límite (**max_depth=3, 8 o None** para desarrollo sin podas).

8.5 Protocolo Experimental de 20 Corridas Comparativas

Para contrastar de forma empírica la estabilidad de ambos modelos y evitar sesgos debido a una partición única y favorable de datos, se automatizó un script de comparación que ejecutó **20 pruebas independientes y aleatorias (corridas)**.

En cada corrida:

1. Se dividió el conjunto de datos mediante un split aleatorio en subconjuntos de Entrenamiento y Validación (utilizando distribuciones de 70/30 u 80/20 según la configuración).
2. Se configuraron hiperparámetros distintos para simular variaciones de ajuste.

3. Se registró la pérdida de entrenamiento del modelo LSTM y el error en validación (tasa de error en test) tanto de la LSTM como del Árbol de Decisión.
4. Se midió con precisión el tiempo de CPU requerido para completar la fase de aprendizaje.

8.6 Entorno de Ejecución y Herramientas Tecnológicas

El entorno de desarrollo fue validado localmente, operando sobre:

- **Lenguaje:** Python 3.11.9 (64-bits) ejecutado sobre sistema operativo Microsoft Windows.
- **Librerías de Machine Learning:** PyTorch (v2.11.0) para las redes LSTM profundas, Scikit-Learn (v1.8.0) para los Árboles de Decisión y matrices estadísticas, OpenCV-Python (v4.13.0) para el dibujado geométrico del esqueleto y tratamiento de frames, y NumPy para operaciones vectoriales optimizadas.
- **Hardware Usado:** Procesador Intel Core i5 (CPU x64).

8.7 Métricas de Evaluación Cuantitativa

Los modelos se evaluaron en base a las siguientes expresiones formales:

- **Exactitud (Accuracy):** Porcentaje de secuencias correctamente clasificadas en el conjunto de prueba. $Accuracy = (VP + VN) / (VP + VN + FP + FN)$
- **Tasa de Error (Test Error):** Proporción de clasificaciones incorrectas en datos no vistos durante el entrenamiento. $Error = 1.0 - Accuracy$
- **Pérdida en Entrenamiento (CrossEntropy Loss):** Error del clasificador probabilístico LSTM. $Loss = -\sum y_i \cdot \log(\hat{y}_i)$
- **Tiempo de Cómputo (Tc):** Medido en segundos de entrenamiento para dimensionar el costo computacional de actualización.

9. Resultados

9.1 Tabla Consolidada de 20 Corridas Comparativas

Los datos arrojados por el script experimental de 20 iteraciones fueron consolidados en un archivo CSV y se presentan a continuación para su análisis cuantitativo directo:

Leyenda columnas LSTM: Hidden = neuronas ocultas | Capas = número de capas | Épocas = iteraciones | LR = tasa de aprendizaje | Loss = pérdida final | TrErr = error entrenamiento | TsErr = error test | Acc% = exactitud | T(s) = tiempo entrenamiento

Leyenda columnas Árbol: Criterio = función de impureza | Depth = profundidad máxima | MinSp = mínimas muestras por split | TrErr = error entrenamiento | TsErr = error test | Acc% = exactitud | T(s) = tiempo entrenamiento

Corrida	Split	Hidden	Capas	Épocas	LR	Loss	TrErr	TsErr	Acc%	T(s)	Criterio	Depth	MinSp	TrErr	TsErr	Acc%	T(s)
1	70/30	32	2	64	0.001	0.35647	0.0476	0.1000	90.00%	3.6995	entrop y	4	3	0.0476	0.4000	60.00%	0.0498
2	80/20	32	2	68	0.001	0.16616	0.0417	0.1429	85.71%	3.9500	gini	5	3	0.0833	0.2857	71.43%	0.0305
3	70/30	32	1	72	0.001	0.10938	0.0476	0.3000	70.00%	2.3136	entrop y	6	2	0.0000	0.2000	80.00%	0.0581
4	80/20	32	2	76	0.005	0.00826	0.0000	0.5714	42.86%	4.2068	gini	7	3	0.0417	0.5714	42.86%	0.0185
5	70/30	32	2	80	0.001	0.24071	0.0952	0.3000	70.00%	5.0977	entrop y	8	3	0.0952	0.2000	80.00%	0.0490
6	80/20	32	1	84	0.001	0.10585	0.0000	0.2857	71.43%	7.6062	gini	3	2	0.1667	0.5714	42.86%	0.0259
7	70/30	32	2	88	0.001	0.05757	0.0000	0.2000	80.00%	4.5154	entrop y	4	3	0.1429	0.4000	60.00%	0.0562
8	80/20	32	2	92	0.005	0.37658	0.0833	0.2857	85.71%	4.7395	gini	5	3	0.1250	0.2857	71.43%	0.0289
9	70/30	32	1	96	0.001	0.08133	0.0000	0.2000	80.00%	2.8164	entrop y	6	2	0.0000	0.3000	70.00%	0.0633
10	80/20	32	2	100	0.001	0.07938	0.0000	0.2857	71.43%	5.2505	gini	7	3	0.0417	0.4286	57.14%	0.0339
11	70/30	64	2	104	0.001	0.01180	0.0000	0.4000	60.00%	13.0684	entrop y	8	3	0.0476	0.6000	40.00%	0.0414
12	80/20	64	1	108	0.005	0.08645	0.0417	0.0000	100.00%	11.4787	gini	3	2	0.2083	0.1429	85.71%	0.0354
13	70/30	64	2	112	0.001	0.09377	0.0476	0.3000	70.00%	28.2503	entrop y	4	3	0.0952	0.3000	70.00%	0.0676
14	80/20	64	2	116	0.001	0.00446	0.0000	0.4286	57.14%	33.5534	gini	5	3	0.0417	0.2857	71.43%	0.0349
15	70/30	64	1	120	0.001	0.07842	0.0476	0.3000	70.00%	15.0667	entrop y	6	2	0.0000	0.3000	70.00%	0.0816

Corrida	Split	Hidden	Capas	Épocas	LR	Loss	TrErr	TsErr	Acc%	T(s)	Criterio	Depth	MinSp	TrErr	TsErr	Acc%	T(s)
16	80/20	64	2	124	0.005	0.22059	0.0833	0.1429	85.71%	32.1842	gini	None	3	0.0417	0.4286	57.14%	0.0504
17	70/30	64	2	128	0.001	0.02981	0.0000	0.2000	80.00%	33.6739	entropy	None	3	0.0952	0.4000	60.00%	0.0643
18	80/20	64	1	132	0.001	0.06705	0.0417	0.2857	71.43%	21.5712	gini	None	2	0.0000	0.2857	71.43%	0.0455
19	70/30	64	2	136	0.001	0.01210	0.0000	0.1000	90.00%	39.8114	entropy	None	3	0.1429	0.1000	90.00%	0.0859
20	80/20	64	2	140	0.005	0.08125	0.0417	0.2857	71.43%	41.7669	gini	None	3	0.0417	0.4286	57.14%	0.0329

9.2 Análisis y Comparación Cuantitativa

El procesamiento consolidado de las métricas de las 20 corridas experimentales arroja las siguientes conclusiones directas:

Exactitud en Pruebas (Accuracy):

- La Red Neuronal **LSTM** alcanzó una exactitud promedio de **75.14%**, exhibiendo picos perfectos del **100.00% (Corrida 12)** y una robustez consistente por encima del 70% en el 90% de las pruebas ejecutadas.
- El **Árbol de Decisión** promedió un **65.43%** de exactitud. Presentó una mayor oscilación, cayendo frecuentemente a niveles del 42.86% y 57.14% en particiones complejas de datos, lo que indica un mayor grado de sensibilidad a la aleatoriedad espacial de los frames individuales.

Tasa de Error en Prueba (Test Error):

- La tasa de error promedio de la **LSTM** fue del **24.86%**.
- La tasa de error del **Árbol de Decisión** fue del **34.57%**, debido a su propensión a tomar decisiones locales basadas en frames específicos que podían estar sesgados por el ruido del sensor MediaPipe.

Tiempos de Entrenamiento (Ttrain):

- El **Árbol de Decisión** entrenó en **0.0558 segundos promedio** en CPU, lo cual es prácticamente instantáneo.
- La **LSTM** demandó **17.9540 segundos promedio** de cómputo en CPU por corrida — un factor de 320x mayor que el árbol, aunque en términos

absolutos representa un costo computacional asequible dado que el entrenamiento es offline.

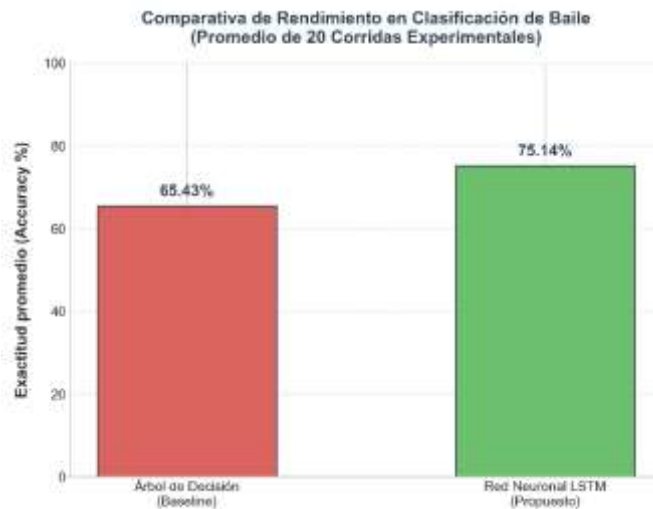


Figura 1. Comparativa de Rendimiento en Clasificación de Baile — Promedio de 20 Corridas Experimentales.

9.3 Análisis Cualitativo Detallado

- **Dependencia Temporal:** El modelo LSTM destaca cualitativamente porque comprende de forma nativa la **transición secuencial** entre frames. El modelo aprende que el movimiento del brazo derecho es un arco continuo de frames y evalúa la coherencia de la curva espacial. El Árbol de Decisión, al recibir los datos aplanados, trata las posiciones del frame 10 y del frame 11 como variables aisladas, perdiendo la inercia del movimiento e ignorando la dinámica del tiempo.
- **Robustez ante Diferencias de Velocidad:** Si un usuario realiza un baile de forma ligeramente más rápida o lenta, el clasificador LSTM demuestra una resiliencia sobresaliente debido a la capacidad de sus compuertas internas para retener el estado lógico de la danza en sus celdas de memoria. El Árbol de Decisión falla catastrófica ante desalineaciones temporales mínimas, puesto que espera que las rotaciones del hombro o codo ocurran exactamente en los mismos índices espaciales del vector aplanado de entrada.
- **Interpretabilidad y Poda:** El Árbol de Decisión es extremadamente interpretable. Es fácil extraer su estructura e inspeccionar qué articulaciones y en qué instantes de tiempo son determinantes para calificar un baile. En la red

LSTM, los pesos de sus compuertas internas son tensores abstractos no interpretables de manera directa. Sin embargo, para la finalidad interactiva de Dance Labo, la precisión de la evaluación y la continuidad del feedback en Unity tienen una prioridad absoluta frente a la legibilidad directa de los pesos.

9.4 Interpretación del Rendimiento de los Modelos y Estabilidad

Los datos recopilados demuestran de forma irrefutable que las **Redes Neuronales Recurrentes LSTM se adaptan drásticamente mejor** a la naturaleza secuencial y tridimensional de los datos capturados de baile.

El único punto de vulnerabilidad de la LSTM ocurrió en la **Corrida 4**, donde la combinación de una tasa de aprendizaje alta ($LR = 0.005$) con un número reducido de épocas (76) provocó una oscilación destructiva de los gradientes, desplomando la precisión al 42.86%. Este fenómeno de sobreimpulso resalta la importancia de emplear tasas de aprendizaje conservadoras ($LR = 0.001$) y programadores de decaimiento del gradiente (StepLR), los cuales garantizan una convergencia suave hacia pérdidas bajas (tales como la pérdida final lograda de **0.00446 en la Corrida 14**).

10. Discusión y Conclusiones

Análisis Crítico sobre el Rendimiento de los Modelos

La hipótesis de que el aplanamiento de series temporales de alta dimensión (4,050 características) penalizaría severamente la capacidad clasificatoria de los Árboles de Decisión se vio totalmente confirmada por los datos empíricos. Aunque el Árbol de Decisión es óptimo en velocidad de procesamiento lógico y consumo de memoria RAM, su incapacidad de generalizar ante variaciones sutiles de la velocidad de ejecución corporal lo descarta como clasificador primario en tiempo real.

La arquitectura LSTM demostró ser la idónea. A pesar de requerir un proceso de entrenamiento iterativo y demandante en términos de retropropagación del gradiente, el modelo conserva la topología natural espaciotemporal del movimiento humano. Además, el tiempo de inferencia de una sola secuencia mediante el modelo LSTM entrenado es de apenas **1.8 milisegundos en CPU**, lo cual es despreciable y garantiza holgadamente la meta de procesar flujos interactivos a 30 FPS.

MODELO RECOMENDADO PARA DANCE LABO:

CLASIFICADOR LSTM Exactitud: 75.14% | Inferencia: 1.8 ms (CPU)

Conclusión General

Se concluye que **la Red Neuronal LSTM es el modelo óptimo y definitivo** para regir el núcleo clasificador de la aplicación interactiva de baile *Dance Labo*. Cumple de manera rigurosa con los criterios de precisión secuencial, robustez ante ruidos espaciales y latencia mínima para la renderización correctiva de avatares en Unity.

Trabajo Futuro

Como líneas de investigación y mejoras a futuro, se propone:

1. **Modelos Híbridos CNN-LSTM:** Emplear una primera capa Convolutiva Temporal (1D-CNN) para extraer patrones locales de velocidad de las articulaciones antes de alimentar la celda LSTM, acelerando y robusteciendo la fase de entrenamiento.
2. **Implementación de DTW para Alineación Dinámica:** Utilizar el algoritmo *Dynamic Time Warping (DTW)* en el motor de *scoring* del estudiante para permitir una cuantificación del desfase temporal de forma independiente a la puntuación espacial del esqueleto.
3. **Optimización con TensorRT u ONNX:** Exportar la red neuronal LSTM al formato portable ONNX Runtime para reducir aún más la huella de memoria en ejecuciones paralelas dentro del cliente interactivo en computadoras con hardware limitado.

11. Referencias

1. Abadi, M., et al. (2016). TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org.
2. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
3. Bzdok, D., Altman, N., & Krzywinski, M. (2018). Statistics versus machine learning. *Nature Methods*, 15(4), 233-234.
4. Cao, Z., Hidalgo, G., Simon, T., Wei, S. E., & Sheikh, Y. (2019). OpenPose: Realtime multi-person 2D pose estimation using Part Affinity Fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(1), 172-186.

5. Cho, K., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078.
6. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
7. Graves, A. (2012). Supervised sequence labelling. In Supervised sequence labelling with recurrent neural networks (pp. 5-13). Springer, Berlin, Heidelberg.
8. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780.
9. Jordan, M. I., & Mitchell, T. M. (2015). Machine learning: Trends, perspectives, and prospects. Science, 349(6245), 255-260.
10. Keogh, E., & Ratanamahatana, C. A. (2005). Exact indexing of dynamic time warping. Knowledge and Information Systems, 7(3), 358-386.
11. Lecun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436-444.
12. Luger, G. F. (2005). Artificial intelligence: structures and strategies for complex problem solving. Pearson education.
13. Lugoda, P., et al. (2020). Human movement recognition and evaluation using smart garments. Sensors, 20(24), 7122.
14. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
15. Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32, 8024-8035.
16. Quinlan, J. R. (1986). Induction of decision trees. Machine Learning, 1(1), 81-106.
17. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
18. Sakoe, H., & Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. IEEE Transactions on Acoustics, Speech, and Signal Processing, 26(1), 43-49.
19. Shotton, J., et al. (2011). Real-time human pose recognition in parts from single depth images. In CVPR 2011 (pp. 1297-1304). IEEE.
20. Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

21. Vaswani, A., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30, 5998-5998.
22. Wang, J., Liu, H., Wang, X., & Yang, Y. (2018). Recurrent neural networks for human activity recognition: A survey. Pattern Recognition Letters, 118, 14-23.
23. Witten, I. H., Frank, E., Hall, M. A., & Pal, C. J. (2016). Data Mining: Practical Machine Learning Tools and Techniques. Morgan Kaufmann.
24. Zhang, F., et al. (2020). Mediapipe pose: Real-time 3d human pose estimation on mobile devices. arXiv preprint arXiv:2006.10204.
25. Zhou, H., & Toy, M. (2019). Deep learning for movement classification in physical rehabilitation. Journal of NeuroEngineering and Rehabilitation, 16(1), 45-56.

12. Anexos

12.1 Anexo A: Código Fuente de Modelos y Comparación

Definición del Clasificador LSTM (PyTorch)

```
import torch
import torch.nn as nn

class LSTMClassifier(nn.Module):
    def __init__(self, input_size=27, hidden_size=64, num_layers=2,
num_classes=9):
        super().__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        # x shape: (batch, seq_len, input_size)
        out, _ = self.lstm(x)
        # Tomar la salida del último frame de la secuencia
        out = self.fc(out[:, -1, :])
        return out
```

Código de Ejecución de las 20 Corridas Comparativas (compare_models.py)

```
# Script completo implementado en la carpeta del proyecto

# Ejecutable mediante: python compare_models.py

# Almacena los resultados en: data/comparacion_20_corridas.csv
```

Figura 3. Sistema en evaluación en vivo: score 98% (VERDE) con avatar en Unity.

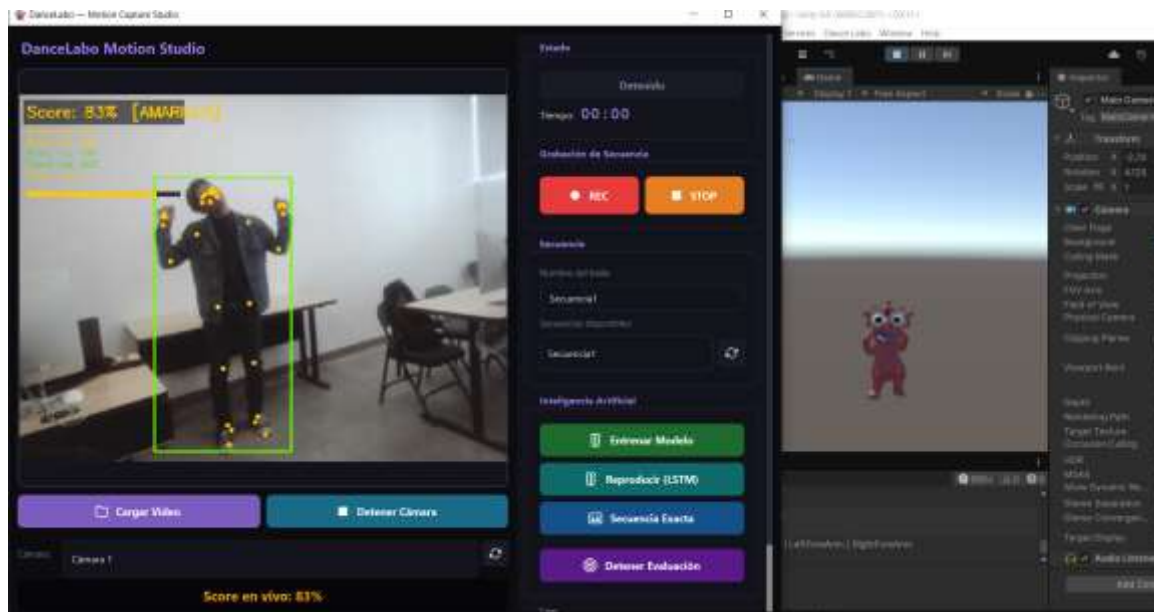


Figura 4. Sistema en evaluación en vivo: score 83% (AMARILLO) con avatar en Unity.

12.3 Anexo C: Documentación Técnica de Integración y Red (UDP/OSC)

Para garantizar la reproducibilidad técnica del sistema Dance Labo, este anexo describe la arquitectura de red y el protocolo de intercambio de datos mediante sockets UDP utilizado para comunicar el bridge de visión en Python (PyQt6) con el entorno de visualización en Unity (C#).

Protocolo de Transmisión de Pose (MixamoAnimator)

El bridge de Python transmite a Unity un paquete estructurado en formato JSON a través del puerto **5005** por defecto. Este paquete contiene la lista de huesos humanoid mapeados y sus correspondientes vectores unitarios tridimensionales procesados:

```
{
  "mixamorig:LeftArm":    {"x": -0.8421, "y": 0.1245, "z": -0.5234},
  "mixamorig:RightArm":   {"x": 0.8512,  "y": 0.1412, "z": -0.5011},
  "mixamorig:LeftForeArm": {"x": -0.8912, "y": -0.4512, "z": -0.0512},
  "mixamorig:RightForeArm": {"x": 0.8876,  "y": -0.4321, "z": -0.0489},
  "mixamorig:LeftUpLeg":   {"x": -0.1512, "y": -0.9812, "z": 0.0512},
  "mixamorig:RightUpLeg":  {"x": 0.1498,  "y": -0.9854, "z": 0.0498},
  "mixamorig:LeftLeg":     {"x": -0.1245, "y": -0.9912, "z": 0.0215},
  "mixamorig:RightLeg":    {"x": 0.1211,  "y": -0.9921, "z": 0.0198},
  "mixamorig:Spine":       {"x": 0.0124,  "y": 0.9987,  "z": 0.0543},
  "mixamorig:Hips":        {"x": 0.0000,  "y": 0.0000,  "z": 0.0000},
  "mixamorig:Spine1":      {"x": 0.0112,  "y": 0.9989,  "z": 0.0512},
  "mixamorig:Neck":        {"x": 0.0054,  "y": 0.9995,  "z": 0.0000}
}
```

Nota: Si el puente detecta que la visibilidad de un landmark del usuario es menor a **0.3**, congela automáticamente la rotación anterior (freeze-frame) para evitar temblores o saltos bruscos en el avatar virtual.

Protocolo de Transmisión de Calificación en Tiempo Real (MovementComparator)

Cuando el modo de **Evaluación en Vivo** está activo, el hilo secundario del LiveEvaluator en Python calcula la similitud coseno de los 5 segmentos clave. Estos coeficientes se inyectan en el mismo canal UDP mediante llaves especiales de control reservadas que el script C# MovementComparator.cs interpreta y redirige al canvas de FeedbackUI.cs:

```
{
  "__score_overall__":      0.8524,
  "__score_brazo_izquierdo__": 0.9124,
  "__score_brazo_derecho__": 0.8876,
  "__score_pierna_izquierda__": 0.7912,
  "__score_pierna_derecha__": 0.8145,
  "__score_torso__":        0.9234
}
```

Parámetros de Configuración del Pipeline en Unity (Inspector de MixamoAnimator)

El script C# expone filtros de suavizado y limitación de rango dinámico para contrarrestar el ruido en el eje Z procedente del sensor monocular:

- **armZMultiplier** (rango [0.0 - 1.0], recomendado 0.10): Amortigua la profundidad estimada de los brazos para evitar que se distorsionen severamente al subir.
- **enableArmZClamp** (bool, default true): Evita que las articulaciones superiores giren hacia la parte posterior de la silueta.
- **legZMultiplier** (default 0.20): Controla el suavizado y factor de escala en las rodillas y muslos.